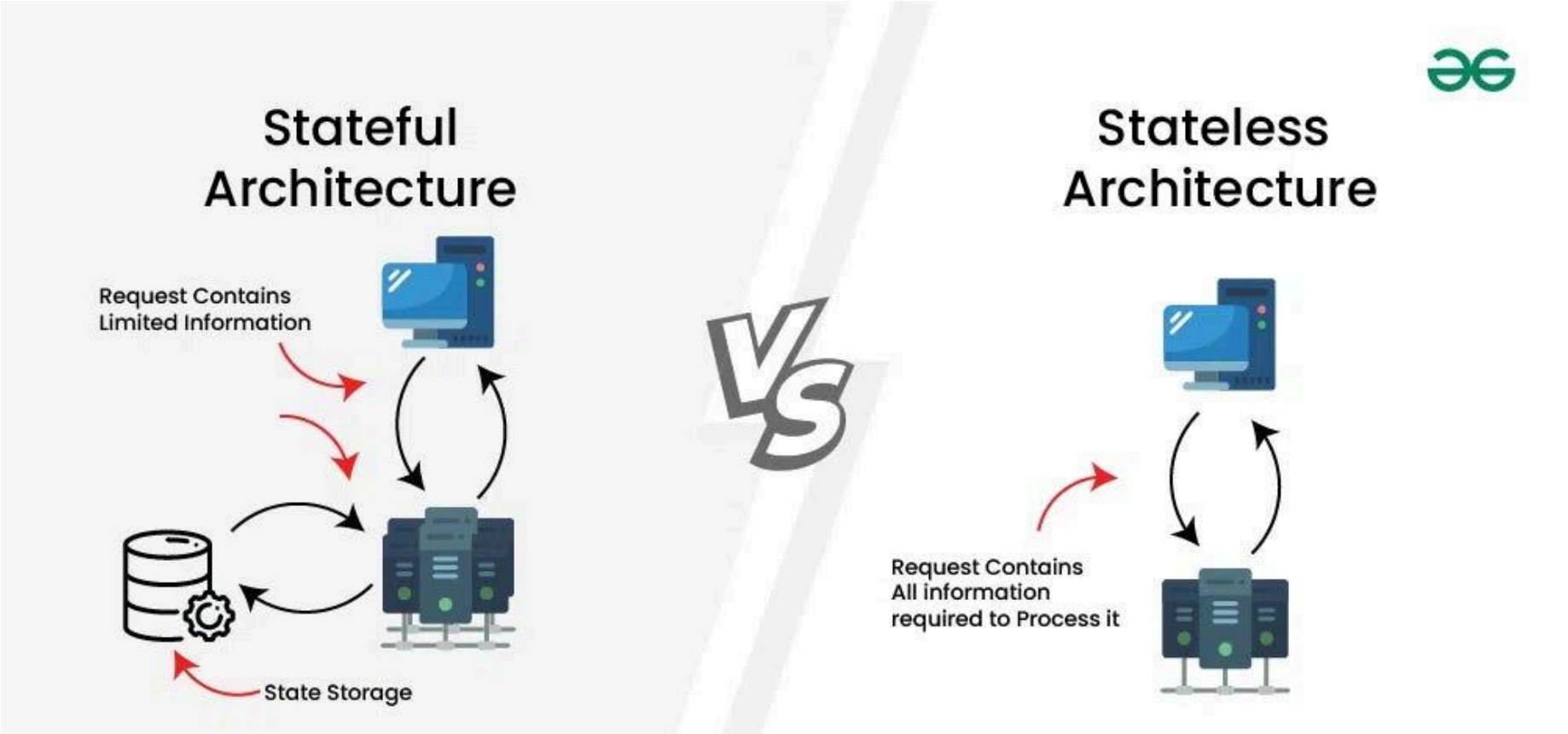
9. Stateful vs Stateless



What Stateful Means

- A system is stateful when it remembers information about past interactions
- The server holds context about the client between requests
- Each new request is not independent — it depends on what happened before
- The server needs to know "who you are" and "what you were doing" to serve you correctly

Real world analogy

- Think of a bank teller who remembers you walked in, showed your ID, and is now mid-transaction
- If you walk away and come back, the teller picks up exactly where you left off
- The teller is holding your state — your identity, your transaction progress

Technical example

- A traditional online game server
- Server tracks your position, health, inventory, session
- Every message from your client is processed in the context of all that stored state
- If the server crashes, your state is lost or must be recovered from a database
- If you get routed to a different server, that server has no idea who you are

Characteristics of stateful systems

- Server stores session data in memory or a local store
- Requests from the same client must go to the same server instance — this is called sticky sessions
- Scaling horizontally is hard because state is tied to a specific server
- If that server goes down, the state is gone unless explicitly persisted elsewhere
- Examples — FTP connections, traditional TCP sessions, multiplayer game servers, stateful firewalls

---

What Stateless Means

- A system is stateless when the server holds zero memory of past interactions
- Every request is completely self-contained
- The request carries all the information the server needs to process it — authentication, context, data
- The server processes the request and forgets it immediately
- From the server's perspective, every request could be coming from a brand new client

Real world analogy

- Think of a vending machine
- You put in money, press a button, get your item

- The machine does not remember you from last time
- Every interaction is fresh and independent
- The machine does not care if it is your first visit or your hundredth

Technical example

- A REST API endpoint GET /products/123
- The server receives this request, fetches the product from the database, returns it, done
- It does not remember this client, does not store anything about this interaction
- The next request from the same client is treated identically to a request from anyone else
- The client must include its auth token on every single request because the server remembers nothing

Characteristics of stateless systems

- Server stores nothing about the client between requests
- Any server instance can handle any request — no sticky sessions needed
- Horizontal scaling is trivial — add more servers, load balancer routes to any of them
- If a server crashes, another one picks up the next request seamlessly
- Examples — HTTP, REST APIs, DNS queries, stateless JWT authentication

---

Key	Stateless	Stateful
Definition	Stateless Protocol is a network protocol in which Client send request to the server and server response back as per the given state.	Stateful Protocol is a network protocol in which if client send a request to the server then it expects some kind of response, in case of no response then it resend the request.
Examples	HTTP (Hypertext Transfer Protocol), UDP (User Datagram Protocol), DNS (Domain Name System)	FTP (File Transfer Protocol), Telnet.
Server Restriction	The Stateless Protocol does not need the server to save any session information.	The Stateful Protocol necessitates that the server saves the status and session data.
Dependency	The server and client in a stateless system are loosely connected and can behave independently.	In Stateful, the server and the client are tightly bound.
Design	Server design is simple to implement. It is very easy to implement a Stateless Protocol on the Internet.	Server design is comparatively complex and it is very difficult to implement a Stateful Protocol on the Internet.
Crash Proof	After a crash, a failed server can be readily resumed.	A server must store session information as well as other details. It's challenging to deal with a crash.
Transactions	Transactions are processed much more quickly by the server.	In comparison, the server is slow.

HTTP Being Stateless by Design

- HTTP was designed from the ground up to be stateless
- Every HTTP request is a completely independent transaction
- The protocol itself carries no memory of previous requests
- There is no built-in concept of a session, a user, or continuity in HTTP

Why it was designed this way

- The original web was built to serve documents
- You request a page, you get it, connection closes

- No need for the server to remember anything
- This design makes the protocol simple, lightweight, and massively scalable
- Any server can handle any request without needing shared memory about past interactions

The consequence of statelessness in HTTP

- The server cannot distinguish between two requests from the same user
- Both requests look identical at the protocol level unless the client explicitly identifies itself
- This is why authentication tokens, cookies, and session IDs exist — they are workarounds layered on top of a stateless protocol to simulate continuity

---

How Sessions and Cookies Simulate State

- HTTP is stateless but real applications need state — login sessions, shopping carts, preferences
- Sessions and cookies are the mechanism used to fake statefulness on top of a stateless protocol
- The state itself lives somewhere (server memory, database, Redis) — cookies just carry the key to find it

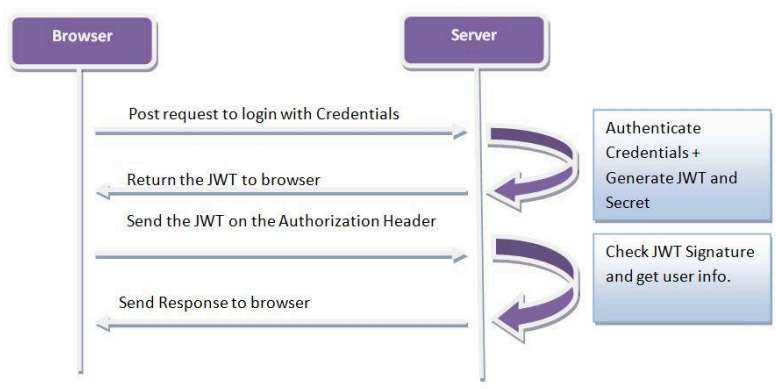
The flow step by step

- User submits login form with username and password
- Server validates credentials
- Server creates a session object and stores it — either in memory, a database, or a cache like Redis
  - Session object contains: user ID, roles, login time, expiry
  - Session is given a unique random ID — example: sess_a8f3bc9d2e
- Server sends this session ID to the browser via Set-Cookie header
  - Set-Cookie: sessionId=sess_a8f3bc9d2e; HttpOnly; Secure; SameSite=Strict
- Browser stores this cookie
- On every subsequent request to that domain, browser automatically attaches the cookie
  - Cookie: sessionId=sess_a8f3bc9d2e
- Server receives the request, reads the session ID from the cookie
- Server looks up sess_a8f3bc9d2e in its session store
- Finds the user data, knows who this is, processes the request accordingly
- From the user's perspective, they are "logged in" — but really HTTP is still stateless
- The illusion of state is created by the client carrying a key and the server using it to look up stored state

The problem this creates at scale

- If session data lives in one server's memory, all requests from that user must go to that server
- This is called sticky sessions or session affinity
- It creates uneven load — some servers get overloaded while others sit idle
- If that server crashes, all sessions stored in its memory are lost — users get logged out
- Solution 1 — store sessions in a centralised store like Redis that all servers can access
  - Now any server can look up any session — no sticky sessions needed
  - But Redis becomes a critical dependency — if it goes down, no one can authenticate
- Solution 2 — move away from sessions entirely and use stateless tokens like JWT

JWT as a stateless alternative



- Instead of storing session data on the server and giving the client a key
- JWT encodes all the user data directly into the token and signs it cryptographically
- The token lives on the client (usually in memory or localStorage)
- Client sends the token in the Authorization header on every request
- Server validates the signature and reads the user data directly from the token
- Server stores nothing — pure stateless authentication
- Tradeoff — you cannot invalidate a JWT before it expires unless you maintain a token blacklist, which reintroduces state

---

Why Stateless is Preferred at Scale

- This is a core system design principle — understand it deeply

Horizontal scaling

- Stateless servers are interchangeable
- Load balancer can send any request to any server
- To handle more traffic, just add more servers — no coordination needed between them
- Stateful servers are not interchangeable — specific requests must go to specific servers
- Scaling stateful servers requires either sticky sessions (uneven load) or shared state stores (added complexity)

Fault tolerance

- If a stateless server crashes, the load balancer routes to another server
- The next request is handled perfectly — the new server needs no context about the past
- If a stateful server crashes, all in-memory state for that server is lost
- Users lose their sessions, ongoing operations fail, recovery is complex

Simplicity of deployment

- Stateless servers can be deployed, restarted, or replaced without any coordination
- Rolling deployments, canary releases, auto-scaling — all work cleanly with stateless servers
- Stateful servers need careful handling during deployments to avoid losing or corrupting state

No session affinity problems

- With stateless servers and a load balancer, round-robin or least-connections routing works perfectly
- With stateful servers, the load balancer must route each client to the same server every time
- Session affinity creates uneven load distribution and complicates infrastructure

Caching friendliness

- Stateless responses are easier to cache because the same request always produces the same response
- Stateful responses depend on server-side context which may differ per user or per session making them harder or impossible to cache

Microservices architecture

- In a microservices system, a request might be handled by 5 different services
- If each service was stateful and needed to know about previous interactions, coordination between services would be a nightmare
- Stateless services process the request they receive, return a response, done
- No cross-service session management needed

The real world tradeoff

- Pure statelessness is an ideal — real systems always have some state somewhere
- The goal in system design is to push state to dedicated, purpose-built storage systems
  - Authentication state → Redis or JWT
  - Application data → database
  - Cache state → Redis or Memcached

    - File state → object storage like S3
- The application servers themselves remain stateless — they just read and write to these stores
- This separation is what makes large-scale systems like those at Google, Netflix, and Amazon work
- The application tier is stateless and infinitely scalable, the state tier is managed separately with its own scaling and redundancy strategies